Context Engineering

The smallest set of high-signal tokens

Finite attention · Anatomy · Budget
Retrieval · Compaction · Memory · Structure · Isolation · Failure · Evals

Part 1: Foundations

Why context is finite · Anatomy of the window · Budgeting it

Part 2: The levers

Retrieval · Compaction · Memory · Structure · Cognitive tools · Protocols · Isolation

Part 3: Practice

Failure modes · Evaluating context · Putting it together

Notes: ai-engineering/02-context-engineering

PART 1

Foundations

Why context is finite · Anatomy of the window · Budgeting it

The thesis

Context rot starts long before the limit

What fills the window

System prompt altitude

Organize by frequency, stability and enforcement

Stable before dynamic — the cache is a prefix match

Patterns that break the cache (or the model)

The status bar — distill, don't retrieve

The window as an economy

Two rules matter more than any split

Progressive reduction — cut by ablation

Measure what each layer buys

The thesis

> **Find the smallest set of high-signal tokens that maximizes the likelihood of the outcome you want.**

Anthropic's framing. It is a minimization: the instinct with a 1M window is to fill it, and the instinct is wrong.

Context rot starts long before the limit

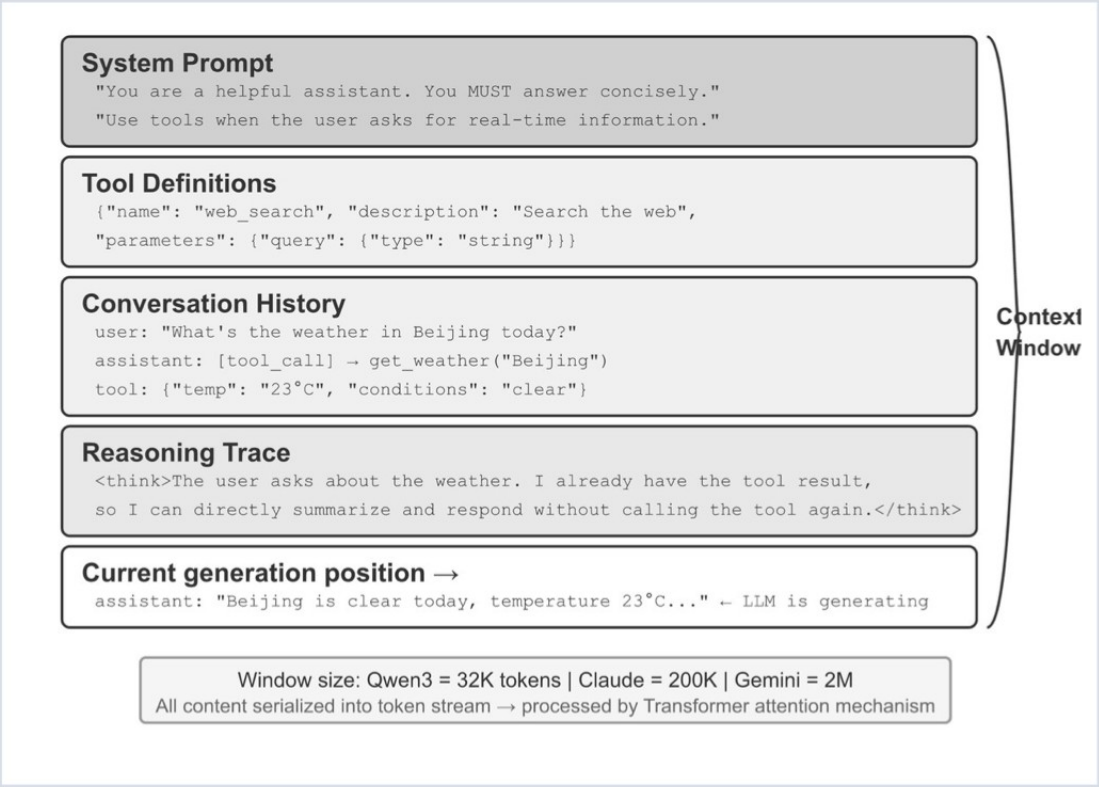
Recall and use of information decline gradually as occupancy grows. It is not a cliff at the window edge.

Shape	What it looks like
Middle neglect	Start and end recalled, the middle is not (lost in the middle)
Distractor sensitivity	Plausible-but-wrong content pulls the answer further off target as the window grows
Instruction decay	System-prompt rules obeyed early in a session are quietly dropped later
Needle degradation	A planted fact, present **verbatim**, is missed more often as filler grows

> **Filler degrades retrieval of the signal it surrounds. "Just in case" is a cost, not free insurance.**

Source: notes/01-why-context-is-finite.md

What fills the window



Li, AI Agents in Depth (2026), Fig. 2-1 (Apache-2.0)

- **System prompt** — role, framing, constraints
- **Tool definitions** — resident every call
- **Retrieved knowledge** and **memory**
- **Runtime state** — date, user, permissions
- **History** — including tool calls and results
- **The current turn** — last

Context engineering decides, per call, what occupies each slot and at what size.

Source: notes/02-context-anatomy.md

System prompt altitude

The Goldilocks zone

Too low	Right altitude	Too high
Hardcoded if/else logic in prose	Specific enough to guide, flexible enough to generalize	Vague guidance that assumes context the model lacks
Brittle on any case not enumerated	Heuristics with clear signals	The model invents its own interpretation
"If the user says X, reply Y. If Z, reply W…"	"Prefer X when the signal is A; escalate when uncertain."	"Be helpful and use good judgment."

> **Adding a fourth conditional to a prompt? The fix is usually a tool or a code-level control, not a fifth.**

Source: notes/02-context-anatomy.md

Organize by frequency, stability and enforcement

always resident

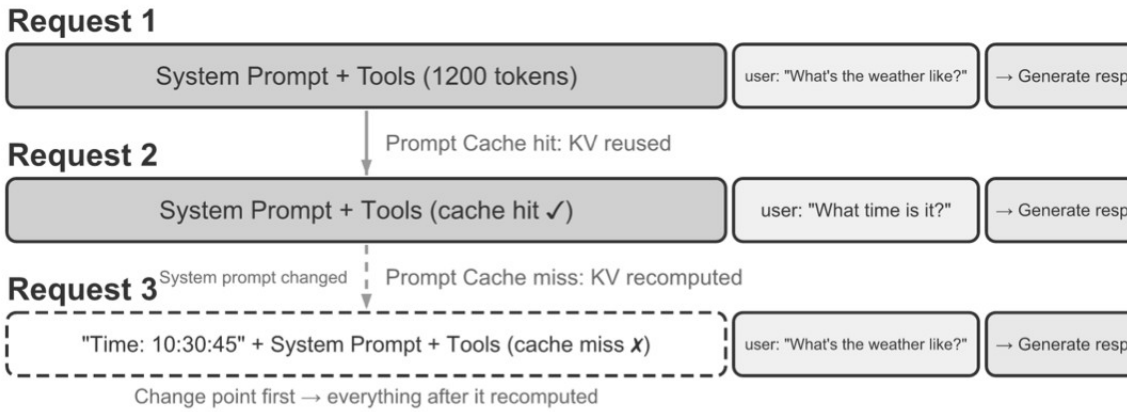
1. **Persistent instruction** — identity, conventions, invariants. Test: true in nearly every task? (`CLAUDE.md`)
2. **On-demand knowledge** — skills and playbooks. ~10 tokens of description resident, the procedure loaded when triggered
3. **Runtime injection** — date, tenant, permissions, assembled at request time rather than stored in the prompt
4. **Long-term memory** — a compact index plus retrievable detail. Never a transcript pasted forward
5. **Deterministic system** — hooks, permissions, schemas, validators. A code control can't be talked out of enforcing itself

enforced in code

> **"The agent keeps doing X" is usually a hook, not a stronger sentence.**

Source: notes/02-context-anatomy.md

Stable before dynamic — the cache is a prefix match



Performance comparison (3000-token total context)

	Cache hit	Cache miss
TTFT *	~0.5 seconds	3–5 seconds
Cost	Only new tokens billed	All tokens after the change rebilled

* TTFT is illustrative; it varies by model, hardware, and deployment and only indicates relative scale.

Li, AI Agents in Depth (2026), Fig. 2-10 (Apache-2.0)

- The first differing token invalidates **everything after it**
- Order: system → tools → stable memory, then retrieved → history → state → query
- Cached reads cost about **1/10** of fresh prefill
- Long documents above the query: up to **30%** better (Anthropic)

> **Cache economics and attention placement point the same way. Design for both from the start.**

Source: notes/02-context-anatomy.md

Patterns that break the cache (or the model)

Li, Experiment 2-3

One timestamp in a 100k-conversation/day system prompt: TTFT **0.5 s → 3–5 s**, and the bill nearly doubled.

Pattern	What breaks	Fix
Timestamp in the system prompt	Prefix changes every request	Late message or a clock tool
Per-request user config (quota, balance)	Same	State at the end (status bar)
Reordering tools by usage	Hundreds of tokens re-prefilled per tool	Fixed order
Sliding-window history	Prefix shifts; old results vanish and calls repeat	Compaction, not truncation
Flattening to `USER:` / `ASSISTANT:` text	Leaves the trained format; tool results ignored	Structured API roles
Per-request few-shot swaps	Examples sit early and break the prefix	A fixed set per task type

Source: notes/02-context-anatomy.md

The status bar — distill, don't retrieve

Li & Shi, 11 models, ~24k evals


```xml
<agent_status>
- phone_call: 3 invocations (Xfinity: 3/3 max)
- Current time: 2025-09-14 10:30:45
- TODO: [1] Cancel plan (in_progress)
</agent_status>
```

- Weak models gain **40–54 points**; a 2B model with a bar matches a frontier model without
- Strong models cut reasoning tokens **80–90%**
- **Maintain it in code**: a 20-line regex beat an LLM summarizer
- Lossy: pairs-only bar on a triple question, **100% → 7.6%**
- Errors above ~10% are worse than no bar

> **In-context learning is closer to retrieval than reasoning. Give the model the count; don't make it count.**

Source: notes/02-context-anatomy.md

The window as an economy

Three things to spend on

Spend	Buys	Over-spent	Under-spent
Instructions — prompt, rules, examples	Behaviour you can rely on	Diluted attention; rules contradict	The model guesses
Evidence — docs, tool results, memory	Grounding	Distraction, rot	Hallucination
Working space — history, reasoning, output	Room to think and answer	Lost recent turns	Truncated output

Investments (a rule used every call) go in the stable prefix. Expenses (a tool result read once) get cleared.

Source: notes/03-budgeting-the-window.md

Two rules matter more than any split

Starting splits (pyramid, by task type) are heuristics. Measure, then adjust.

A starting split — complex reasoning

- System **15%** · Examples **25%**
- History **30%** · Input **20%**
- Reserve **10%**
- Simple Q&A shifts to 70% history, 0% examples

A guess to measure against

The rules

- **Reserve the output first** — `max_tokens` plus the thinking budget come off the top
- **Give every growing component a ceiling** — history, tool results, retrieved chunks
- The ceiling is what triggers compaction

A full window leaves no room to answer

Source: notes/03-budgeting-the-window.md

Progressive reduction — cut by ablation

Before (~350 tokens)

You are a financial analyst with a PhD from Stanford and 15 years at Goldman Sachs and Morgan Stanley. You specialise in technology sector analysis with deep knowledge of SaaS business models…

Persona biography

After (~90 tokens)

You are a senior financial analyst specialising in tech stocks. Cover: 1. Fundamentals 2. Industry context 3. Macro factors. Balance quantitative data with strategic insight.

Task guidance, three sections

Remove anything whose deletion doesn't move an eval score. Few-shot returns fall fastest after the first example.

Source: notes/03-budgeting-the-window.md

Measure what each layer buys

PYTHON

```python
layers = {"role": ..., "examples": ..., "constraints": ..., "audience": ...}
prompt = "Write a paragraph about climate change."
for name, text in [("base", "")] + list(layers.items()):
    prompt = f"{text}\n\n{prompt}" if text else prompt
    out = llm(prompt)
    p, r = len(enc.encode(prompt)), len(enc.encode(out))
    print(f"{name:12} prompt={p:4} response={r:4} score={judge(out):.2f}")
```

> **Track quality per prompt token. Response/prompt length ("token efficiency" upstream) rewards verbosity.**

Source: notes/03-budgeting-the-window.md

PART 2

The levers

Retrieval · Compaction · Memory · Structure · Cognitive tools · Protocols · Isolation

Retrieve before inference, or during it
The pre-retrieval pipeline — cheap filters first
A tuned pipeline — Bayer's PRINCE
Hybrid retrieval and reranking
Contextual retrieval — add context at index time
A first production cut
RAG architectures — each step adds control
The compaction pipeline
What survives compaction, in order
Six compression strategies, one task
The production ladder — cheapest first
Memory formats — mix them
Memory hygiene — update over append
Designed memory vs. searching the raw record
Choosing a format
Output schemas — validate, and feed the error back
Cognitive tools — reasoning steps as isolated calls
Verify with code where you can
Protocol shells — prompt programs without code
Isolation — the most expensive lever
Tool descriptions are resident context

Retrieve before inference, or during it

The biggest shift in the pillar

Pre-computed (RAG era)

- Embed corpus and query, stuff top-k, generate
- Relevance guessed **before** the model reasons
- 50 files × 5k tokens = **250k**

Fast, deterministic, often wrong

Just-in-time (agentic)

- Keep **identifiers**: paths, IDs, queries (~10 tokens each)
- The agent loads what its reasoning says it needs
- Index + 3 files = **~15.5k**

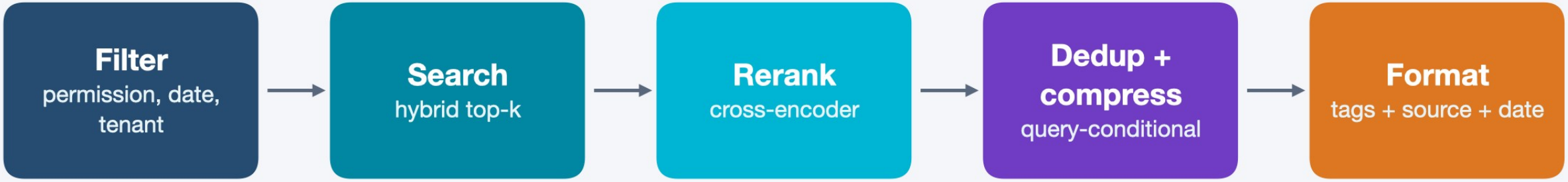
Costs: latency, wandering, non-determinism

Hybrid default: pre-load what's needed on ≥80% of turns; leave the long tail to tools.

Source: notes/04-retrieval.md

The pre-retrieval pipeline — cheap filters first

When you do retrieve upfront



- **Filter inside the vector query**, not on the results. It's also a security control
- **Ranking ≠ retrieval** — retrieve wide, rerank, send narrow
- **Dedup trap** — a chunk hit by several sub-queries is often the most relevant one
- **Compress per query** — a generic summary drops the detail the query needed

Source: notes/04-retrieval.md

A tuned pipeline — Bayer's PRINCE

millions

1. **Extract keywords + a metadata filter** — two concurrent LLM passes, e.g. `eq(study_id, T123456-2)`
2. **Expand the query, n=5** — a smaller model rewrites the question into five variants
3. **Pre-filter inside the vector DB** — millions of vectors → tens or hundreds before any similarity math
4. **Parallel hybrid search, 0.7 / 0.3** — kNN **0.7**, keyword **0.3**, found experimentally; union, max-score, keep ~20
5. **Cross-encoder rerank, 20 → 7** — `bge-reranker-large` against the **original** question, not the rewrites

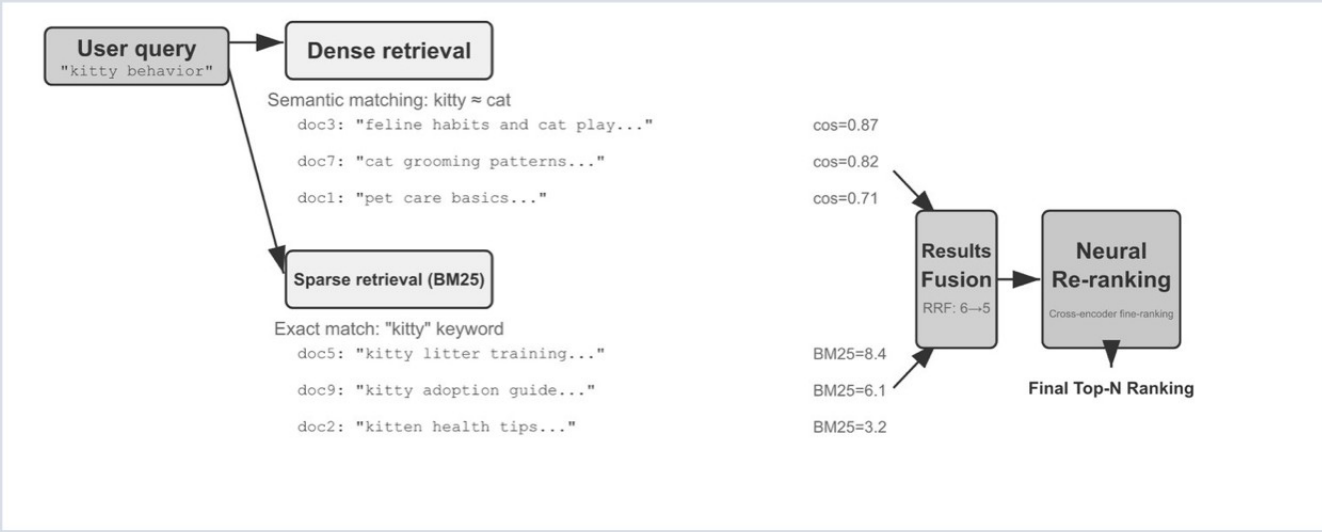
7 chunks

Each narrowing uses a mechanism more accurate and more expensive than the last.

Source: notes/04-retrieval.md

Hybrid retrieval and reranking

The production default

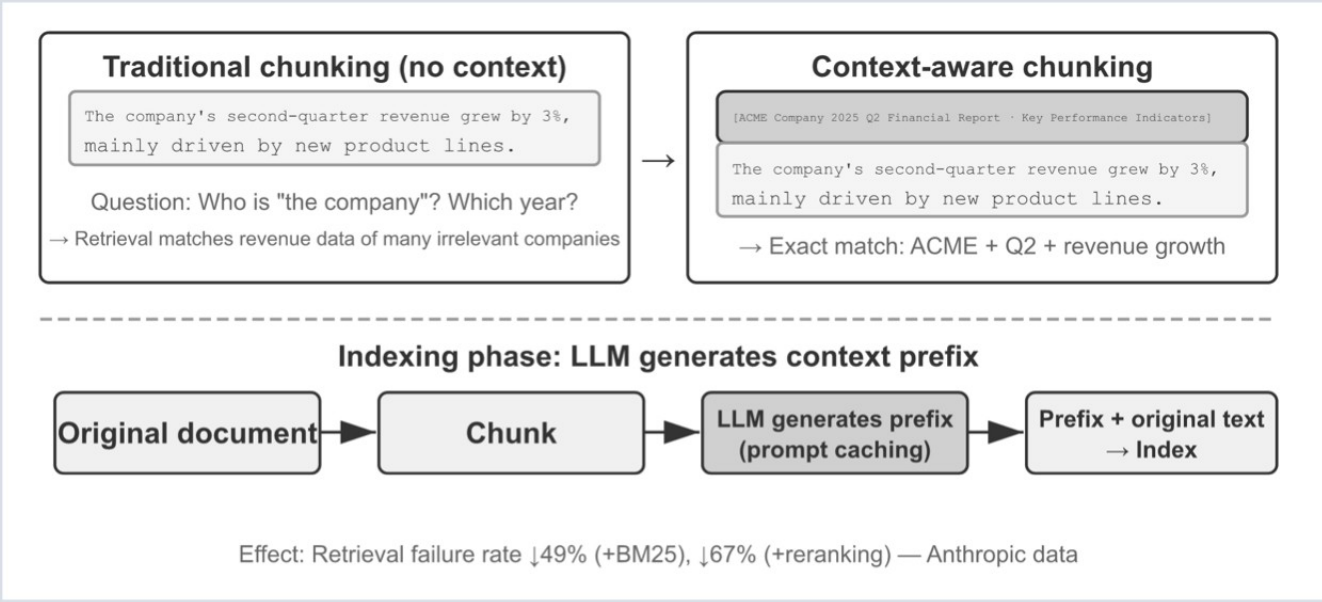


Li, AI Agents in Depth (2026), Fig. 3-9 (Apache-2.0)

- **BM25** finds exact terms: names, error codes, IDs
- **Dense** finds paraphrases
- **Fuse**: RRF `Σ 1/(60 + rank)` is robust; weighted scores need normalizing first
- **Rerank** the fused top ~50 with a cross-encoder
- Measure each stage: **recall@k**, MRR, nDCG

Source: notes/04-retrieval.md

Contextual retrieval — add context at index time



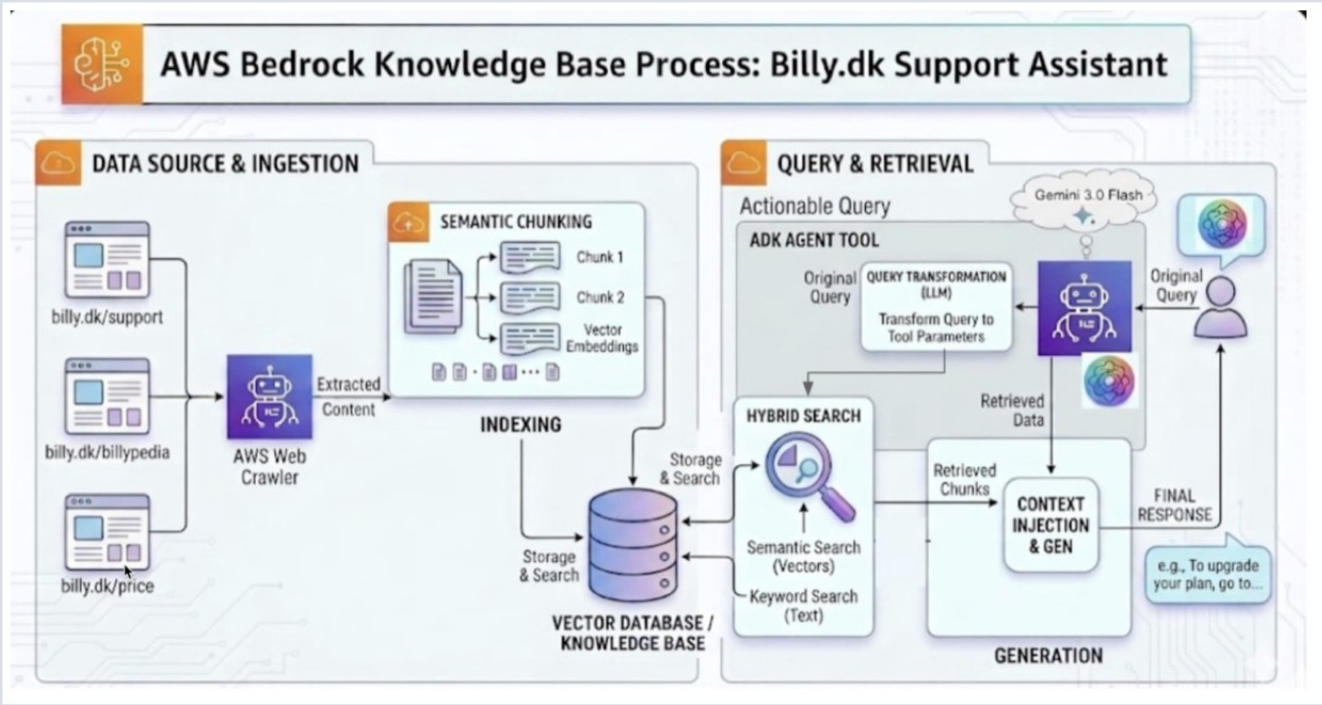
Li, AI Agents in Depth (2026), Fig. 3-14 (Apache-2.0)

- "Q2 revenue grew 3%" — which company, which year?
- An LLM writes a short prefix per chunk; index prefix + chunk
- Top-20 failures **–49%** (contextual embeddings + BM25)
- **–67%** with a reranker
- About **$1 per million** document tokens with prompt caching

Contextual retrieval adds at index time. Contextual compression removes at query time. Don't confuse them.

A first production cut

Worked example



Billy.dk support assistant on AWS Bedrock Knowledge Bases

- **Hybrid search**: semantic and keyword together
- **Query transformation** before retrieval: what users type rarely retrieves best
- Missing: **no rerank, no dedup**
- Those are the stages teams add after quality plateaus

Source: notes/04-retrieval.md

RAG architectures — each step adds control

From naive to agentic

Architecture	Shape	Use when
Naive	embed → top-k → stuff into prompt	One-hop questions over a clean corpus
Modular	rewrite → retrieve → rerank → compress → generate	You need to swap one stage
Agentic	model decides whether, what and when; loops	Multi-step questions that decompose
Graph-enhanced	Entities and relations; retrieval walks edges	Multi-hop, "how is X related to Y"
Text-to-SQL	schema subset + retrieved few-shot + SELECT-only check	Counting, filtering, aggregation

> **A layer spanning structured and unstructured data needs a router, not a single index.**

Source: notes/04-retrieval.md

The compaction pipeline



- **Trigger at phase boundaries**, not at 95% mid-edit; ~80% occupancy as a backstop
- **Identifiers verbatim**: UUIDs, hashes, paths, PR numbers. One wrong digit fails the next call

- **Tool-result clearing first**: highest value, lowest risk
- Compaction rewrites the prefix, so each one pays a full re-prefill

Source: notes/05-compression-compaction.md

What survives compaction, in order

Retention priority

1. **Architectural decisions** — "X over Y because Z", verbatim. Lose the reason and the agent re-litigates it
2. **Modified files and critical changes** — the diff so far is state, not history
3. **Verification status** — which tests ran, pass or fail
4. **Unresolved TODOs and rollback notes** — everything not yet done
5. **Tool output** — deletable. Keep the conclusion, drop the payload

> **40k tokens of test output → "17 passed, 2 failed". A rationale can't compress at all.**

Source: notes/05-compression-compaction.md

Six compression strategies, one task

Li, Experiment 2-9

Track OpenAI co-founders' current roles with a 128K window. Seven searches return ~367k characters.

Strategy	Result
No compression	Overflowed at iteration 5. Failed
Summarize each result separately	12 iterations, 277k tokens; facts repeated across pages
One combined summary	10 iterations, 93k tokens; long inputs truncated at the tail
Context-aware (query + known state in the prompt)	**7 iterations, 40k tokens**; a 148k-char page → 2k, every role change kept
Context-aware + citations	223k tokens; lossy content, lossless index to the sources
Adaptive: batch at 80%, `[COMPRESSED]` marker	175k tokens; full fidelity early on

> **Compression is understanding. The compressor needs near-main-model capability and the task in its prompt.**

Source: notes/05-compression-compaction.md

The production ladder — cheapest first

Claude Code's five mechanisms

1. **Tool-result budget** — large outputs go to disk; the model sees a frozen preview
2. **Delete noise outright** — summarizing junk wastes tokens
3. **API micro-compaction** — server-side context editing; still breaks the cache after the edit point
4. **Archival per-turn summaries** — `git log`, not `git squash`
5. **Full LLM compaction** — last resort, with a circuit breaker for compaction-failure loops

> **Crash recovery: past ~30 minutes of runtime, write progress to disk. State on disk survives everything.**

Source: notes/05-compression-compaction.md

Memory formats — mix them

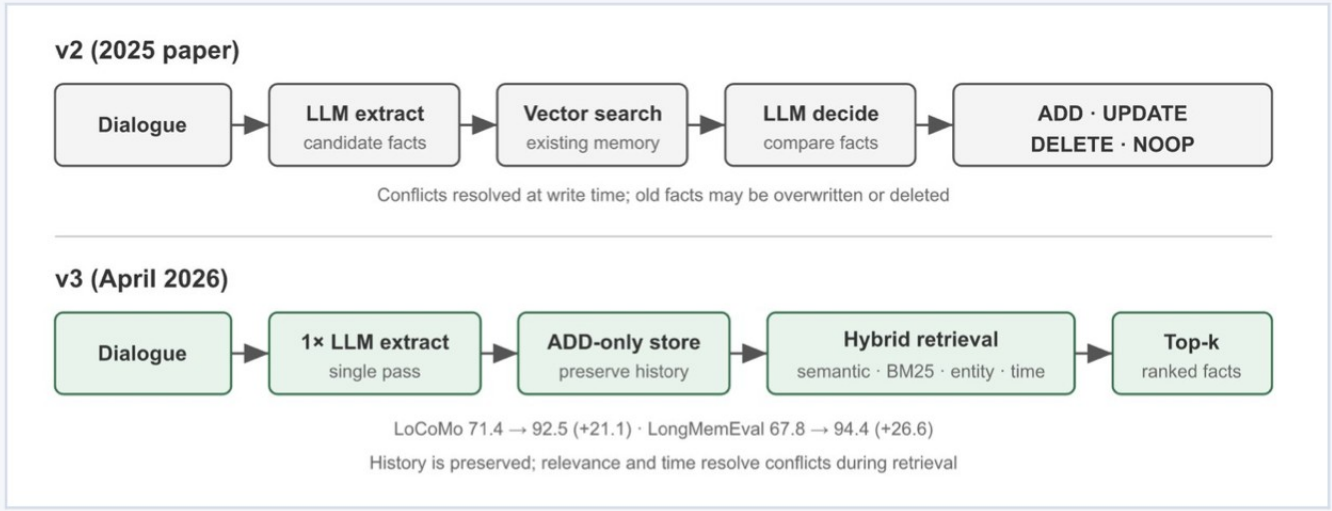
Memory affects the model only by entering a window. Every memory decision is a context decision.

Format	Wins	Loses
Simple notes	Cheapest; passes basic recall	Links between facts (title, employer, team become three facts)
Enhanced notes	Nuance	Redundant; one change rewrites several paragraphs
JSON cards	Partial updates; predictable	Rigid when a fact fits three categories
Advanced JSON cards	Disambiguation: the user's dentist vs. their father's cardiologist	Expensive to generate and maintain
User as Code	Typed state from a fact log: aggregation **~99%** vs. 6–43%	A `notes` field for what won't type

> **Advanced cards for critical, low-volume facts. Simple notes for high-volume, low-stakes ones.**

Source: notes/06-memory.md

Memory hygiene — update over append



Li, AI Agents in Depth (2026), Fig. 3-3 (Apache-2.0)

- Extract candidate facts; vector-match against existing memories
- LLM picks **ADD · UPDATE · DELETE · NOOP**
- "Moved to Shanghai" UPDATEs "lives in Beijing"
- Absolute dates; don't store what's derivable
- Verify a remembered file or flag still exists before acting

> **Append-only memory is slow-motion context poisoning.**

Source: notes/06-memory.md

Designed memory vs. searching the raw record

Peng, arXiv 2610.29823

One LoCoMo conversation: 419 turns, 152 questions.

Extraction-based memory (Mem0, Zep)

- Beats full context only with the weakest reader (GPT-4o-mini, **78% vs. 74%**)
- Ties or loses with strong models; the build alone costs more than a full-context run
- **The writer caps every reader**: Opus 5.5 on a GPT-4o-mini graph, 45% vs. 43%

Raw history plus search tools

- A coding agent reading the raw history as one file: **99%**
- At under half full context's cost
- Same direction as index-plus-detail

Caveats: one conversation, one run per cell

With a strong model, keep the source, make it searchable, and let the model decide what to load.

Source: notes/06-memory.md

Choosing a format

Structure compresses meaning, not tokens. The model spends less attention working out what each part is.

Format	Best for	Watch out for
XML-style tags	Wrapping documents, separating sections	Verbose; keep tag names meaningful
Markdown	Instructions, readable system prompts	Weak boundaries: a retrieved doc's `##` collides with yours
JSON	Machine state, tool I/O	Token-heavy; escaping inside strings
YAML	Config-like nested state	Whitespace; ambiguous types
Tables	Comparable records	Wide tables lose alignment

> **Tags for boundaries, markdown for prose, JSON for anything a program reads back.**

Source: notes/07-structured-context.md

Output schemas — validate, and feed the error back

Without decode-time enforcement

PYTHON

```python
for _ in range(max_attempts):
    raw = llm(prompt)
    fenced = re.search(FENCE_RE, raw, re.DOTALL)
    try:
        text = fenced.group(1) if fenced else raw
        obj = json.loads(text)
        jsonschema.validate(obj, schema)
        return obj
    except (json.JSONDecodeError,
            jsonschema.ValidationError) as e:
        prompt += (f"\n\nYour previous response did not"
                   f" conform. Error: {e}. Try again.")
raise ValueError("no schema-valid response")
```

- Prefer the provider's **structured-output mode**
- Otherwise: schema in the prompt + one filled example
- The validator's message tells the retry what to fix
- Keep `jsonschema.validate` for rules the provider's subset can't express

A malformed state block that gets re-injected compounds every turn. Validate at the boundary.

Source: notes/07-structured-context.md

Cognitive tools — reasoning steps as isolated calls

Ebouky et al., IBM Research 2025

GPT-4.1 on AIME 2024: **26.7%** → **43.3%**, near o1-preview, with no training.

Tool	What it does
`understand_question`	Core concepts, givens, goal, relevant techniques
`recall_related`	Analogous solved problems and their steps
`examine_answer`	Check the trace for errors and unmet constraints
`backtracking`	When stuck, find where it went wrong and try another path

> **Each tool runs in its own context. The model choosing when to call them beat a fixed sequence.**

Source: notes/08-cognitive-tools.md

Verify with code where you can

PYTHON

```python
def python_validator(response):
    blocks = re.findall(CODE_RE, response, re.S)
    if not blocks:
        return False, "No Python code block found."
    for i, block in enumerate(blocks, 1):
        try:
            compile(block, "<llm>", "exec")
        except SyntaxError as e:
            return False, f"Block {i}: {e}"
    return True, "ok"
```

- Loop: generate → validate → feed the problem back, **max 3**
- Same-model verifiers share blind spots; past 2–3 rounds they rarely help
- A **self-graded** stop condition (score > 0.8) is a demo, not a pattern
- Tests, parsers, calculators beat a model verdict

End a model verifier with a machine-readable verdict. Better, replace it with a deterministic check.

Source: notes/08-cognitive-tools.md

Protocol shells — prompt programs without code

PROTOCOL

```
/research.literature{
 intent="Structured understanding of a topic",
 input={ topic="Context compaction, 2024–26",
         key_questions=[...], scope={...} },
 process=[
   /map{action="Foundations, streams, terms"},
   /explore{action="Key works, methods, limits"},
   /synthesize{action="Patterns, open questions"}
 ],
 output={ findings="Per question, cited",
          confidence="High/med/low + reason" }
}
```

- **Real:** explicit intent, inputs, steps and output improve consistency
- **Oversold:** the syntax has no special power; markdown with the same four sections does as well
- One action per step; inputs, not assumptions
- Use it for shareable procedures with non-engineers

Source: notes/09-protocols.md

Isolation — the most expensive lever

Why it works

- 3 sub-agents spend **230k** exploring; the orchestrator sees **~6k**
- The sub-agent window is **discarded**, not merged
- **Orchestrator holds the plan**; workers get a bounded task, minimal context, a return shape

Clean windows for synthesis

What it costs

- Lost shared context; duplicated or contradictory work
- The summary is lossy: what it omits is gone
- Token multiplier: 230k real tokens for 6k visible

Try Compress before Isolate

Sub-agents are for genuinely parallel, separable investigation. Pass the decisions, not just the task.

Source: notes/10-tools-and-multi-agent.md

Tool descriptions are resident context

Bad (~45 tokens)

```
description: |
 This skill handles the complete
 deployment process to production.
 It covers environment checks,
 rollback procedures, and post-
 deploy verification. Use this
 before deploying any code to
 production.
```

Good (~9 tokens)

```
description: Use when deploying to
 production or rolling back.
```

Plus: when NOT to use it, and what instead

Results are context too: paginate, return identifiers, return structured errors.

Source: notes/10-tools-and-multi-agent.md

PART 3

Practice

Failure modes · Evaluating context · Putting it together

- Five ways context fails
- Prompt injection — layer the defenses
- The diagnostic flow
- Context evals instrument each stage
- Ablation — the core context experiment
- The maturity ladder — climb only when you must
- Designing a context system — the order of questions
- Appendix: the speculative field track

Five ways context fails

Failure	Mechanism	Signature
Rot	Occupancy grows; recall degrades	Right earlier, wrong later; facts present but unused
Poisoning	A false statement is treated as ground truth	Confidently builds on something wrong
Distraction	Marginal content crowds out signal	Answers an adjacent question
Clash	Two contradictory pieces coexist	Inconsistent answers; arbitrary resolution
Injection	Adversarial instructions arrive as data	Follows instructions the operator never gave

> **Distinct causes with distinct fixes, routinely misdiagnosed as each other or as "the model isn't smart enough".**

Source: notes/11-failure-modes.md

Prompt injection — layer the defenses

probabilistic

1. **Structural delimiting** — wrap untrusted content in tags; it is data, never instructions
2. **Trust zones** — never promote a zone implicitly
3. **Instruction hierarchy** — the system prompt outranks retrieved content, unconditionally
4. **Isolation** — untrusted content in a discarded sub-agent window; its summary is data too
★ **Code-level enforcement** — a permission hook can't be talked out of denying a write
6. **Egress control** — allowlists, confirmation gates, no credentials in reachable context

guaranteed

> **Every context-level defense is probabilistic. Only code-level controls are guarantees.**

Source: notes/11-failure-modes.md

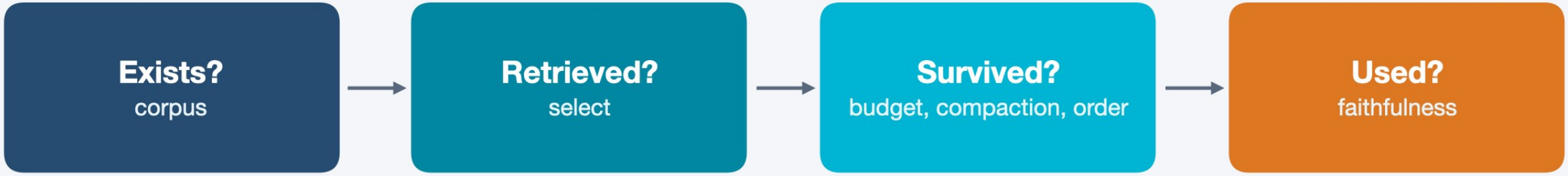
The diagnostic flow

1. **Print the actual window** — what was sent, not what you think you assembled. Most bugs are assembly bugs
2. **Check occupancy** — above ~50%, suspect rot and distraction first
3. **Search for contradictions** — the wrong claim appears twice with different values → clash
4. **Check provenance** — no legitimate source for the wrong fact → poisoning or injection
5. **Test in isolation** — same task, minimal context. Works → context problem. Fails → prompt or capability

> **Step 5 is the one that gets skipped. It separates a context problem from a prompt problem.**

Source: notes/11-failure-modes.md

Context evals instrument each stage



- **Select:** recall@k, precision@k, MRR
- **Use:** faithfulness, context utilisation, citation accuracy. Faithful ≠ correct
- **Compress:** plant N facts, compact, ask for each. A ratio means nothing without retention

- **Long context:** length sweep at 10/25/50/75%; the knee is your budget
- **Cost:** report tokens per successful task next to accuracy

Source: notes/12-evaluating-context.md

Ablation — the core context experiment

Remove one component, hold everything else fixed, re-run the eval set, record Δ quality and Δ tokens.

Δ quality	Δ tokens	Action
≈ 0	large	**Cut it**
Small drop	large	Compress it
Large drop	any	Keep it; protect it from compaction

> **Grow the eval set from failures: every production context bug becomes a regression case.**

Source: notes/12-evaluating-context.md

The maturity ladder — climb only when you must

Level	What's in the window	Chapters
1 · Atom	Task + constraints + output format	02
2 · Molecule	+ examples (few-shot)	02, 03
3 · Cell	+ memory carried across turns	05, 06
4 · Organ	Specialised contexts under an orchestrator	10
5 · Cognitive system	+ reasoning tools, schemas, verification loops	07, 08

> **Most production failures at level 4 are level-2 problems. Each rung multiplies cost and failure modes.**

Source: notes/13-putting-it-together.md

Designing a context system — the order of questions

1. **What does success look like?** — build the eval set first
2. **What's the smallest window that could work?** — start at level 1 or 2
3. **What does the model lack?** — facts → retrieval; continuity → memory; method → cognitive tools
4. **What grows?** — a ceiling and a compaction rule for each
5. **What can live outside the window?** — write before select, select before compress, isolate last
6. **What can go wrong?** — walk the five failure modes against the design

Source: notes/13-putting-it-together.md

Appendix: the speculative field track

Note 99, confidence: low

About a third of the upstream course treats context as a "semantic field". It's a metaphor, not a method.

Vocabulary you'll meet

- **Attractors**, **resonance**, **boundaries**, **quantum semantics**
- "Protocols" like `/attractor.co.emerge` act as ordinary structured prompts
- Real result underneath: symbolic mechanisms (Yang et al., ICML 2025)

What survives translation

- **Early framing is sticky** — restart rather than argue
- **Salience is earned** by structure, edge placement and restating at decision points
- **Ambiguity is resolved by context** — find the underspecified term

Ask what the measurable behaviour is

Source: notes/99-speculative-field-track.md

Rules of thumb

The pillar on one slide

Wrap-up

Assembling

Stable before dynamic. Reserve the output first; give every growing component a ceiling.

Getting content in

Just-in-time via identifiers. Hybrid search + reranker. Tag sources; let the model say "not here".

Keeping it small

Clear tool results first. State in a small overwritten block. Distil episodes into facts.

Reasoning

Understand, solve, verify as separate calls. A deterministic check beats a model verifier.

Isolation

Compress before Isolate. The orchestrator holds the plan; tool outputs are context.

Shipping

Look in the window before the weights. Ablate to set the budget. Climb the ladder only on evidence.

Source: README.md#rules-of-thumb

Where to go next

Wrap-up

- **03 — Harness engineering:** when the loop that assembles context becomes stateful
  - Execution boundaries, orchestration, reliable agents (PRINCE)
- **06 — Eval:** RAG and grounding metrics, tracing where the tokens go
- **Run the code:** the snippets in each note; practice/01 builds BM25 by hand
- **Read:** Anthropic, Effective context engineering for AI agents; Li, AI Agents in Depth ch. 2–3
- **Copy from:** templates/ — schemas, prompt programs, minimal-context YAML

The one idea to keep

Every token competes for a fixed attention budget. Justify each one's place, keep the rest one tool call away, and measure.

Source: README.md